



IT SIMPLERA INSTITUTE

Cyber Security Analyst Internship Program

Week 03 Assignment

Prepared By

Muhammad Kamran Akbar

Submitted To: Senior Cyber Security Analyst

Department: Cyber Security

Submission Date: 18 July 2026

Task 1: Common Network Protocols & Port Numbers

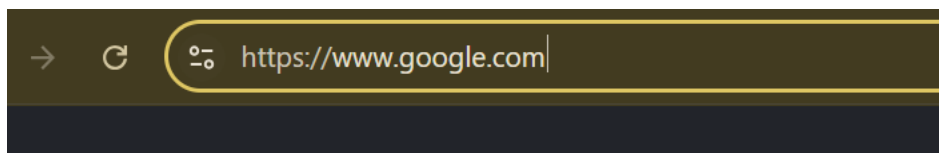
Here are 15 protocols with the details required:

#	Protocol	Port	TCP/UDP	Purpose	Secure/Insecure
1	FTP (Control)	21	TCP	File transfer control channel	Insecure (plaintext credentials)
2	FTP (Data)	20	TCP	File transfer data channel	Insecure
3	SSH	22	TCP	Secure remote login/command execution	Secure (encrypted)
4	Telnet	23	TCP	Remote login	Insecure (plaintext)
5	SMTP	25	TCP	Sending email	Insecure (unless using STARTTLS)
6	DNS	53	TCP/UDP	Domain name resolution	Insecure (plain DNS); DoH/DoT secure
7	DHCP	67/68	UDP	Automatic IP address assignment	Insecure (no built-in auth)
8	TFTP	69	UDP	Simple file transfer (no auth)	Insecure
9	HTTP	80	TCP	Web page transfer	Insecure (plaintext)
10	POP3	110	TCP	Retrieving email from server	Insecure (unless SSL/TLS wrapped)
11	NTP	123	UDP	Time synchronization	Insecure (can be spoofed)
12	IMAP	143	TCP	Email retrieval (keeps mail on server)	Insecure (unless SSL/TLS)
13	SNMP	161/162	UDP	Network device monitoring/management	Insecure (v1/v2c); v3 secure
14	HTTPS	443	TCP	Encrypted web page transfer	Secure (TLS/SSL)
15	RDP	3389	TCP	Remote desktop access	Semi-secure (encrypted but has had vulnerabilities)

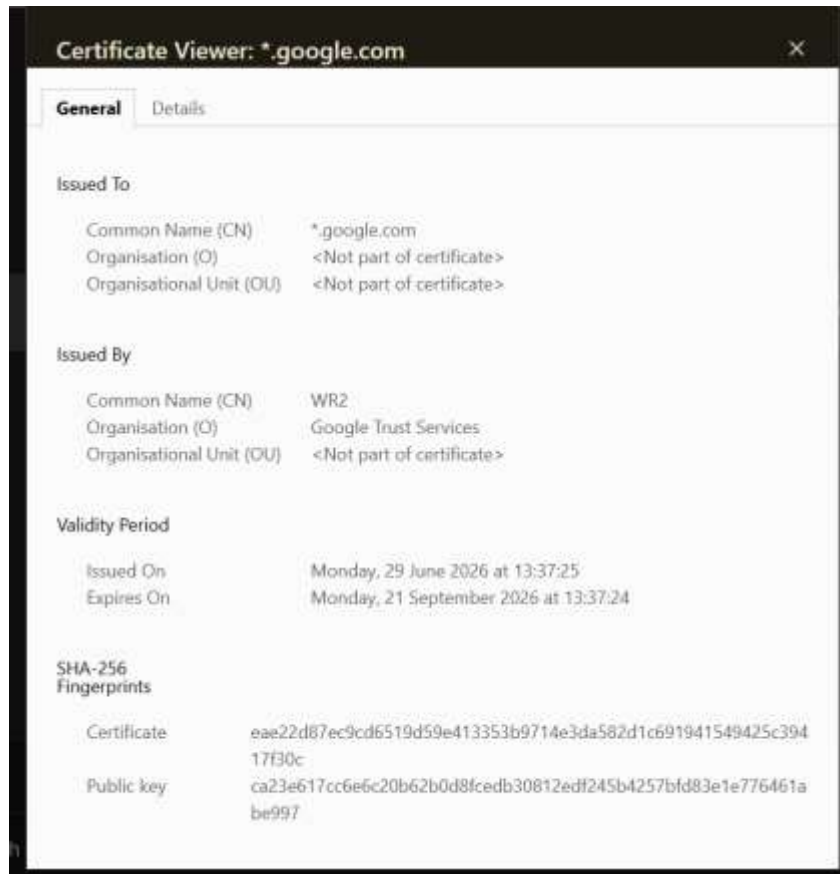
Task 2 : “HTTP vs HTTPS Analysis”

1.Google.com

- Uses HTTPS with a wildcard certificate (*.google.com) issued by Google's own CA (Google Trust Services), and auto-redirects from HTTP to HTTPS.

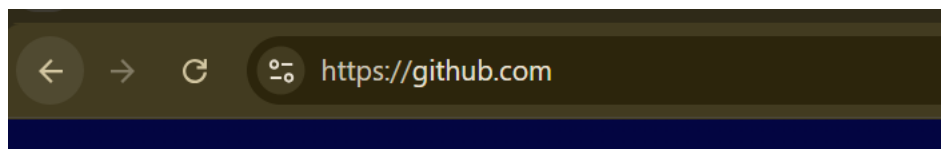


- Certificate has a short ~3-month validity window, which is a modern security practice that limits damage if a certificate is ever compromised.



2.GitHub.com

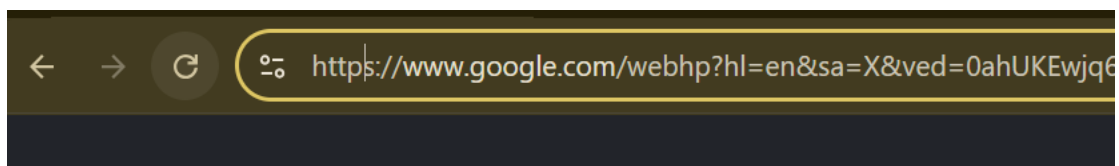
- Uses HTTPS with an exact-match certificate (CN: github.com, not a wildcard), issued by a third-party CA — Sectigo — unlike Google which issues its own certificates.



- Certificate validity is ~3 months (3 July – 1 Oct 2026), showing GitHub also follows the modern short-lifespan certificate practice.



3. Google Gemini





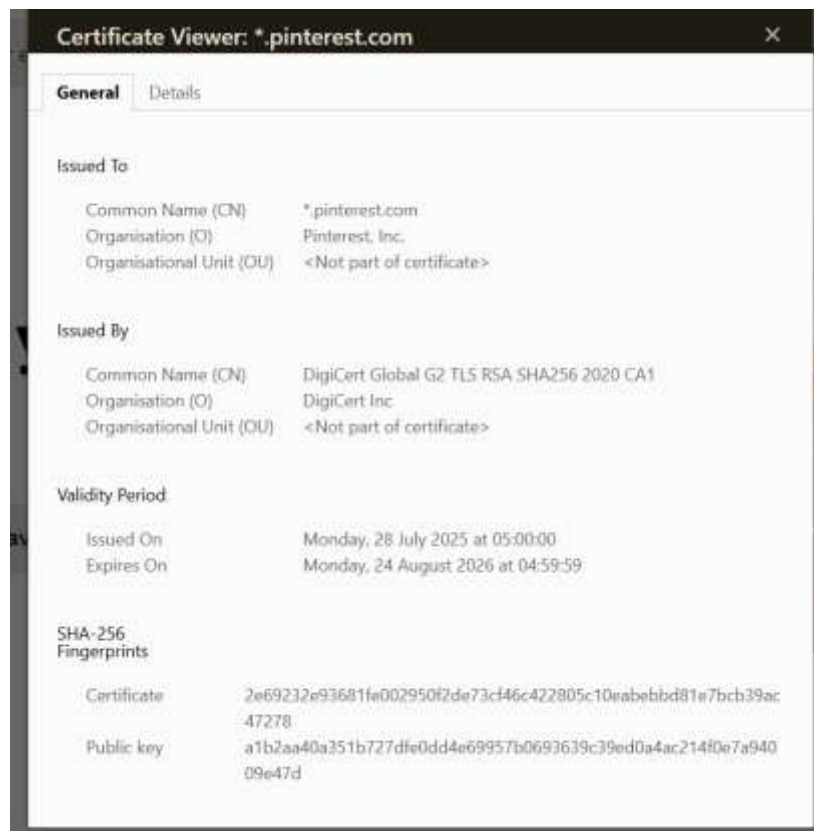
4. Amazon.com

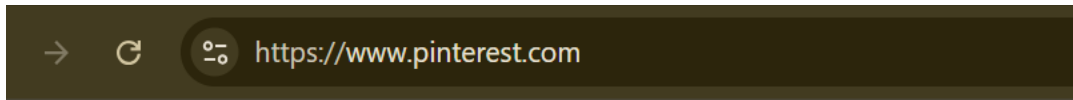


- Uses HTTPS with an exact-match certificate (CN: www.amazon.com), issued by Amazon's own internal CA — "Amazon RSA 2048 M04" — similar to how Google issues its own certs.
- Certificate validity is about 1 year (19 Jan 2026 – 12 Jan 2027), notably longer than Google's and GitHub's ~3-month certificates, showing Amazon follows a different renewal cycle.
- Since Amazon handles payments and personal data at scale, HTTPS here isn't just for privacy — it's essential for protecting checkout/login pages from interception.



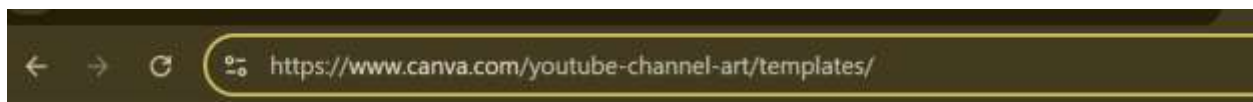
5. Pinterest.com





- Auto-redirects from `http://` to `https://www.pinterest.com`, confirming HTTPS is enforced by default.
- Uses a wildcard certificate (`*.pinterest.com`), unlike Amazon and GitHub's exact-match certs, issued by a third-party CA — DigiCert — rather than Pinterest's own infrastructure.
- Certificate validity spans about 13 months (28 July 2025 – 24 Aug 2026), a longer window than Google/GitHub's ~3-month certs, showing DigiCert-issued certs here follow a more traditional annual renewal cycle.

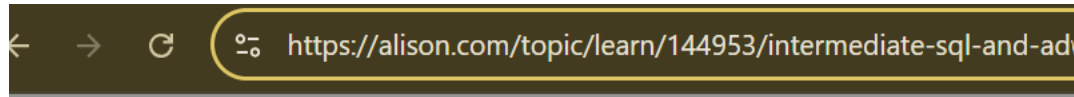
6.Canva.com



- Uses a wildcard certificate (`*.canva.com`), interesting because it's issued by "Amazon RSA 2048 M04" — showing Canva relies on AWS's certificate infrastructure (they're hosted on Amazon Web Services) instead of a dedicated CA like DigiCert.
- Certificate validity is about 13 months (20 Nov 2025 – 20 Dec 2026), similar to Pinterest's longer renewal cycle.



7. Alison.com



- Uses a wildcard certificate (*.alison.com), issued by GoDaddy — a smaller, third-party CA compared to the DigiCert/Amazon-issued certs seen so far, showing GoDaddy is still commonly used by mid-sized platforms.
- Certificate validity is about 13 months (4 March 2026 – 5 April 2027), consistent with the longer renewal cycle we've seen from third-party CAs (similar to Canva and Pinterest)



8. Teams.live.com

- Uses an exact-match certificate (CN: teams.live.com), issued by Microsoft's own internal CA — "Microsoft TLS G2 RSA CA OSCP 16" — similar to how Google and Amazon issue certs for their own services.
- Certificate validity is about 1 year (13 Jan 2026 – 8 Jan 2027), matching the typical annual cycle used by large corporations for internal apps.
- Since Teams is used for meetings, chat, and file sharing in professional/enterprise settings, HTTPS is critical here to protect confidential business communications from interception.



9.Instagram.com



- Uses a wildcard certificate (*.www.instagram.com), issued by DigiCert — owned by Meta Platforms, Inc., same parent company as Facebook.
- Certificate validity is exactly 3 months (26 April – 26 July 2026), matching the short-lifespan pattern used by Google, showing large tech platforms are increasingly favoring frequent cert rotation for stronger security.

10. TikTok.com

- Uses a wildcard certificate (*.www.tiktok.com), issued via "RapidSSL TLS ECC CA G1" — a budget-tier certificate line under DigiCert, notably lighter-weight than the direct DigiCert Global CA used by Instagram/Pinterest.
- Certificate validity is exactly 1 year (13 Oct 2025 – 13 Oct 2026), and uses ECC (Elliptic Curve Cryptography) rather than RSA — a newer encryption approach that offers similar security with smaller key sizes and faster performance.

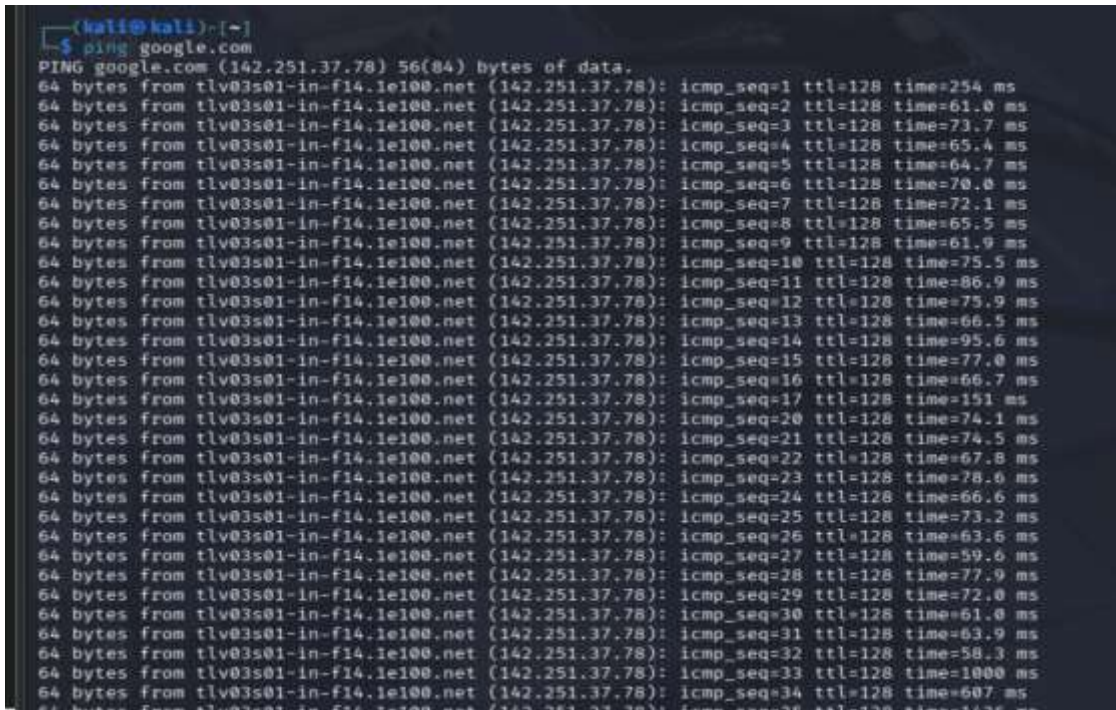


Task 3: Kali Linux Network Reconnaissance

1. Ping Test — google.com

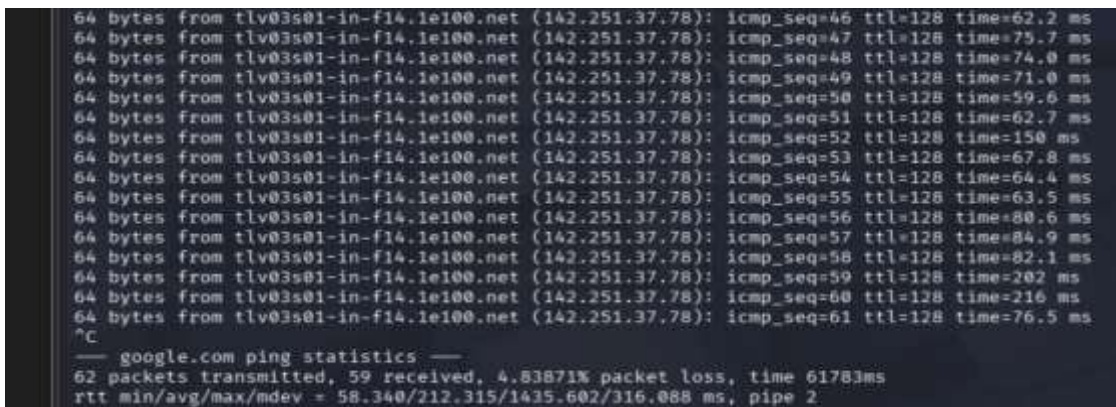
Command used:

```
ping google.com
```



```
(kali@kali)~$ ping google.com
PING google.com (142.251.37.78) 56(84) bytes of data:
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=1 ttl=128 time=254 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=2 ttl=128 time=61.0 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=3 ttl=128 time=73.7 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=4 ttl=128 time=65.4 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=5 ttl=128 time=64.7 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=6 ttl=128 time=70.0 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=7 ttl=128 time=72.1 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=8 ttl=128 time=65.5 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=9 ttl=128 time=61.9 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=10 ttl=128 time=75.5 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=11 ttl=128 time=86.9 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=12 ttl=128 time=75.9 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=13 ttl=128 time=66.5 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=14 ttl=128 time=95.6 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=15 ttl=128 time=77.0 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=16 ttl=128 time=66.7 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=17 ttl=128 time=151 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=20 ttl=128 time=74.1 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=21 ttl=128 time=74.5 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=22 ttl=128 time=67.8 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=23 ttl=128 time=78.6 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=24 ttl=128 time=66.6 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=25 ttl=128 time=73.2 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=26 ttl=128 time=63.6 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=27 ttl=128 time=59.6 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=28 ttl=128 time=77.9 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=29 ttl=128 time=72.0 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=30 ttl=128 time=61.0 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=31 ttl=128 time=63.9 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=32 ttl=128 time=58.3 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=33 ttl=128 time=1000 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=34 ttl=128 time=607 ms
```

Figure 1a: Ping output showing ICMP replies from google.com



```
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=46 ttl=128 time=62.2 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=47 ttl=128 time=75.7 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=48 ttl=128 time=74.0 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=49 ttl=128 time=71.0 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=50 ttl=128 time=59.6 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=51 ttl=128 time=62.7 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=52 ttl=128 time=150 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=53 ttl=128 time=67.8 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=54 ttl=128 time=64.4 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=55 ttl=128 time=63.5 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=56 ttl=128 time=80.6 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=57 ttl=128 time=84.9 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=58 ttl=128 time=82.1 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=59 ttl=128 time=202 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=60 ttl=128 time=216 ms
64 bytes from tlv03s01-in-f14.1e100.net (142.251.37.78): icmp_seq=61 ttl=128 time=76.5 ms
^C
— google.com ping statistics —
62 packets transmitted, 59 received, 4.83871% packet loss, time 61783ms
rtt min/avg/max/mdev = 58.340/212.315/1435.602/316.088 ms, pipe 2
```

Figure 1b: Ping summary statistics

Observations

- TTL Value: 128 — this is the default TTL for Windows-based systems, and it stayed consistent across all replies, indicating no route changes during the test.
- Response Time: mostly ranged between 58 ms and 90 ms, which is a normal, healthy response time for a request travelling to Google's servers.

- A few requests spiked significantly higher (up to 1435 ms), which is likely caused by temporary network congestion, Wi-Fi interference, or momentary ISP-side delay rather than an issue with Google's servers.
- Packet Loss: 4.83% (59 of 62 packets received) — a small amount of packet loss is common over Wi-Fi and does not indicate a serious connectivity problem, though a wired connection would typically show 0% loss.
- Overall, the connection to google.com is stable, with occasional latency spikes rather than a consistent slow connection.

2. Nmap Scan — scanme.nmap.org

Command used:

`nmap scanme.nmap.org`

```
(kali@kali)~$ nmap scanme.nmap.org
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-18 09:44 EDT
Stats: 0:01:25 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 48.45% done; ETC: 09:47 (0:01:29 remaining)
Stats: 0:01:27 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 48.53% done; ETC: 09:47 (0:01:31 remaining)
Stats: 0:01:28 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 48.55% done; ETC: 09:47 (0:01:32 remaining)
Stats: 0:01:28 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 48.57% done; ETC: 09:47 (0:01:32 remaining)
Stats: 0:01:29 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 48.58% done; ETC: 09:47 (0:01:33 remaining)
Stats: 0:03:16 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 73.78% done; ETC: 09:48 (0:01:09 remaining)
```

Figure 2: Nmap SYN stealth scan in progress against scanme.nmap.org

Observations

- The scan against scanme.nmap.org did not complete within the available time window, reaching about 74% completion during the SYN Stealth Scan phase before it had to be stopped.
- scanme.nmap.org is a public test target intentionally provided by the Nmap project for practising scans, and it is common for scans against it to take longer than scans on a local network, since traffic must travel across the internet and the host may rate-limit or delay probe responses from many simultaneous users.
- This is a valid and expected finding to report: it demonstrates that scan duration depends heavily on network distance and target-side conditions, not just the scanning tool itself.

3. Nmap Scan — Local Machine

Command used to identify and scan the local subnet:

`nmap -A 192.168.12.128/24`

```

(kali@kali)~$ nmap -A 192.168.12.128/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-18 09:49 EDT
Stats: 0:00:36 elapsed; 252 hosts completed (3 up), 3 undergoing Script Scan
NSE Timing: About 0.00% done
Nmap scan report for 192.168.12.1
Host is up (0.00076s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
135/tcp   open  msrpc  Microsoft Windows RPC
2179/tcp  open  vmrpd?
MAC Address: 00:50:56:C0:00:08 (VMware)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running: (JUST GUESSING) Microsoft Windows 11|10|2008 (91%), FreeBSD 6.X (88%)
OS CPE: cpe:/o:microsoft:windows_11 cpe:/o:freebsd:freebsd:6.2 cpe:/o:microsoft:windows_10_2008
Aggressive OS guesses: Microsoft Windows 11 21H2 (91%), FreeBSD 6.2-RELEASE (88%), Microsoft Windows 10 1607 (85%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

TRACEROUTE
HOP RTT      ADDRESS
1   0.76 ms 192.168.12.1

Nmap scan report for 192.168.12.2
Host is up (0.00053s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
53/tcp    filtered domain
MAC Address: 00:50:56:FB:A9:E2 (VMware)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: specialized
Running: VMware Player
OS CPE: cpe:/a:vmware:player
OS details: VMware Player virtual NAT device
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   0.53 ms 192.168.12.2

```

Figure 3a: Nmap scan of the local subnet (part 1)

```

Host is up (0.00069s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
53/tcp    filtered domain
MAC Address: 00:50:56:FA:39:66 (VMware)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: specialized
Running: VMware Player
OS CPE: cpe:/a:vmware:player
OS details: VMware Player virtual NAT device
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   0.69 ms 192.168.12.254

Nmap scan report for 192.168.12.254
Host is up (0.00069s latency).
All 1000 scanned ports on 192.168.12.254 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:FA:39:66 (VMware)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   0.69 ms 192.168.12.254

Nmap scan report for 192.168.12.128
Host is up (0.00046s latency).
All 1000 scanned ports on 192.168.12.128 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
Too many fingerprints match this host to give specific OS details
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 256 IP addresses (4 hosts up) scanned in 51.81 seconds

(kali@kali)~$

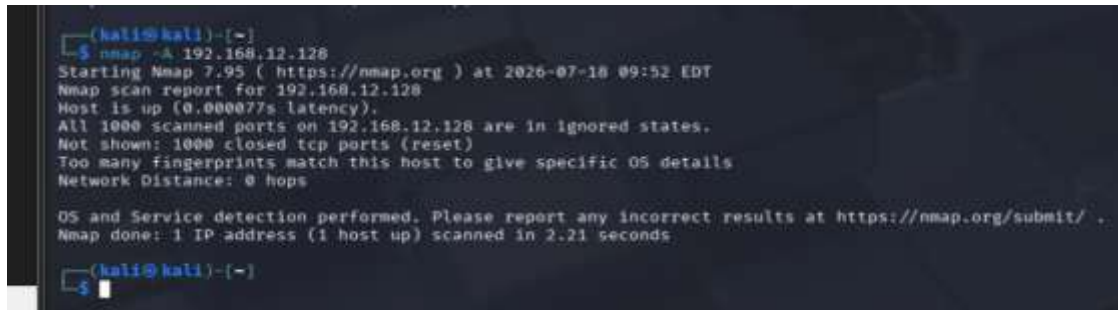
```

Figure 3b: Nmap scan of the local subnet (part 2)

Follow-up scan focused on own IP (192.168.12.128)

Command used:

```
nmap -A 192.168.12.128
```



```
(kali@kali)-[~]
└─$ nmap -A 192.168.12.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-18 09:52 EDT
Nmap scan report for 192.168.12.128
Host is up (0.000077s latency).
All 1000 scanned ports on 192.168.12.128 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
Too many fingerprints match this host to give specific OS details
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.21 seconds

(kali@kali)-[~]
└─$
```

Figure 3c: Focused Nmap scan of the student's own local IP address

Task 4: What Happens When You Type a URL and Press Enter?

When we type something like `www.google.com` into the browser and hit Enter, a lot happens behind the scenes in just a fraction of a second before the page actually shows up.

Step 1: Checking the cache first

The browser doesn't immediately go out to the internet. It first checks if it already knows the IP address for that website from a previous visit. It looks in a few places in order — first the browser's own cache, then the operating system's cache, then the router's cache, and finally the ISP's cache. If the address is found anywhere in this chain, it skips straight to connecting.

Step 2: DNS resolution

If nothing is cached, the computer needs to translate the domain name into an IP address, since computers don't understand names like "google.com" — only numbers. This is where DNS (Domain Name System) comes in. Think of DNS like a phonebook: you know the name, DNS tells you the number. The request goes out to the ISP's DNS server, which either has the answer already or forwards the request further up the chain until the IP address is found.

Step 3: Setting up the connection (TCP handshake)

Now that the browser has the IP address, it needs to actually establish a connection with that server. This is done using TCP (Transmission Control Protocol), through what's called a three-way handshake — the browser sends a SYN (synchronize) request, the server replies with SYN-ACK, and the browser confirms back with an ACK. Once this handshake is complete, a reliable connection is open between the two machines.

Step 4: TLS handshake (if HTTPS)

If the site uses HTTPS, there's one more step before any data is exchanged — the browser and server perform a TLS handshake to agree on encryption keys and verify the server's certificate. This is what makes the padlock appear in the address bar.

Step 5: Sending the HTTP request

Once connected, the browser sends an HTTP request to the server — usually a GET request, asking for the actual page content.

Step 6: Server processes the request

The server receives this request, figures out what's being asked for, and prepares a response. This might involve pulling data from a database, running some backend code, or just fetching a static file.

Step 7: Server sends back the response

The server sends back the requested content, usually as HTML, along with a status code (like 200 OK if everything went fine, or 404 if the page wasn't found).

Step 8: Browser renders the page

Finally, the browser takes the HTML it received and starts rendering it on screen — pulling in any additional resources like CSS, JavaScript, and images along the way, until the full page is visible.

Task 5: Networking Cheat Sheet

Protocols, Ports, and Security Considerations

Protocol	Port	TCP/UDP	Purpose	Security Risk	Secure Alternative
HTTP	80	TCP	Loads web pages between browser and server	Data travels in plaintext, so anyone on the network path can read or alter it	HTTPS (443)
HTTPS	443	TCP	Loads web pages with encryption via SSL/TLS	Weak or outdated TLS versions, or expired certificates, can still leave it exposed	Keep TLS updated, use valid certificates
FTP	21	TCP	Transfers files between systems	Login credentials and file contents sent in plaintext	SFTP (22) or FTPS (990)
SSH	22	TCP	Remote login and command execution, encrypted	Weak passwords make it a target for brute-force attacks	Key-based authentication + MFA
Telnet	23	TCP	Remote terminal access, unencrypted	Everything, including passwords, is sent in plaintext	SSH
SMTP	25	TCP	Sends email between mail servers	Emails can be intercepted or spoofed if unencrypted	SMTP with STARTTLS (587) or SMTPS (465)
DNS	53	TCP/UDP	Resolves domain names into IP addresses	Vulnerable to spoofing and cache poisoning, which can redirect users to fake sites	DNS over HTTPS (DoH) or DNS over TLS (DoT)
DHCP	67/68	UDP	Automatically assigns IP addresses to devices on a network	A rogue DHCP server on the network can hand out malicious configs to devices	DHCP Snooping on switches
TFTP	69	UDP	Simple file transfer with no login required	No authentication or encryption at all	SFTP or SCP
POP3	110	TCP	Downloads email from a mail server	Credentials and messages exposed if sent unencrypted	POP3S (995)
NTP	123	UDP	Synchronizes system clocks across a network	Can be abused for DDoS amplification attacks	Authenticated NTP (NTS)
IMAP	143	TCP	Syncs email between client and server	Traffic unencrypted unless wrapped in TLS	IMAPS (993)
SNMP	161/162	UDP	Monitors and manages network devices	Older versions (v1/v2c) use weak, plaintext community strings	SNMPv3 (adds authentication + encryption)
SMB	445	TCP	Shares files and printers over a network	Past vulnerabilities (e.g. used by WannaCry ransomware) if left	SMBv3 with encryption enabled, regular patching

Protocol	Port	TCP/UDP	Purpose	Security Risk	Secure Alternative
				unpatched	
RDP	3389	TCP	Remote desktop access to another machine	Frequently targeted by brute-force login attempts and ransomware	Enable NLA, use MFA + VPN, strong passwords

